

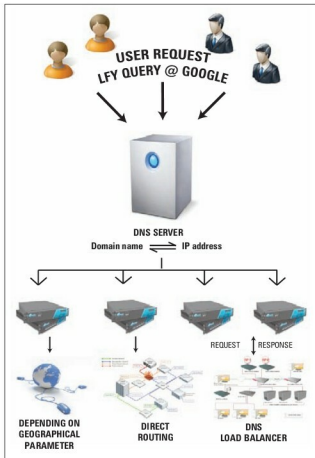


Figure 1: Google's load-balancing architecture

2. **Load balancing through IP tunnelling:** The load balancer just schedules requests to different real servers, and the real servers return replies directly to users. Thus, the load balancer can handle a huge amount of requests without being a bottleneck in the system.
3. **Load balancing through direct routing:** This is similar to the previous category, in which real servers reply on their own, but the difference is that the response can be routed, and need not incur tunnelling overhead.

There are numerous open source load balancers in the industry, which are getting better, day by day, as they are being taken care of by contributors and the community. They outclass the others due to extra features, multi-platform support, low or no licence costs, and their association with other open source-based load balancers. For example, LVS (Linux Virtual Server) and Ultra Monkey, exhibit these characteristics.

Some of the open source load-balancing solutions available in the market are given below (I have tried to prioritise the list):

1. Linux Virtual Server Project
2. Ultra Monkey
3. Balance
4. BalanceNG

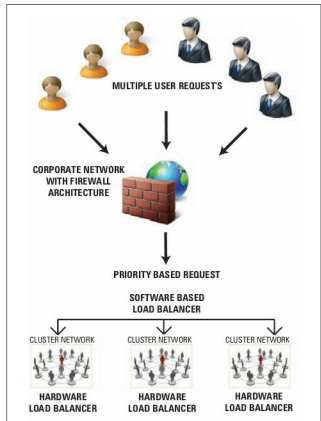


Figure 2: Load balancing in an organisation

5. Red Hat Cluster Suite
6. The High Availability Linux Project

Now, I will explain how to configure LVS for clustering on SuSE, and how LVS acts as a scalable and high-performance delivery platform.

LVS for clustering on SuSE

Figure 3 depicts the intended logical layout of our load-balancer set-up.

LVS, like an ideal load balancer, acts as a pathway to real server configuration. There are both hardware-based and software load-balancing configurations available for LVS. LVS load balances incoming requests, and distributes them across the architecture using some scheduling methods. It uses all three types of load balancing—i.e., NAT, IP tunnelling and direct routing.

A load balancer is a single point of entrance for all requests, so it's really important to have a backup or fail-over ready. In this example, *heartbeat2* is used as a fail-over monitor for the load balancers. *Ldirectord* monitors the health of the real clusters by periodically requesting a known URL, and checking that the response contains an expected string. If a service fails on a server, then the server is taken out of the pool of real clusters, and can be segregated for maintenance.

In this way, LVS maintains two important parameters